COMPUTER SOCIETY OF INDIA
COEP TECH STUDENT CHAPTER

with

Simplify Healthcare

as our

Title Sponsor

presents

INSPIRON 5.0

A NATIONAL LEVEL HACKATHON

in association with

INFOTECH WORKS

as our

Co-Title Sponsor

PROBLEM STATEMENTS

PROBLEM STATEMENT 01

US Healthcare EDI Parser & X12 File Validator (837/835 / 834)

Domain: Healthcare IT / FinTech

The Core Challenge: The US healthcare system processes over 5 billion EDI transactions annually — primarily X12 837 (medical claims), 835 (payment-remittance), and 834 (member enrollment/disenrollment) files. These ASCII-delimited files are notoriously difficult to parse, validate, and audit. Billing teams lose millions to claim rejections caused by malformed segments, and HR/benefits teams face enrollment errors that leave employees without coverage.

Background & Context

Electronic Data Interchange (EDI) is the backbone of US healthcare administration. The ASC X12 standard defines strict transaction sets: 837P/837I (professional and institutional claims submitted by providers to insurers), 835 (Electronic Remittance Advice sent by insurers back to providers after claim adjudication), and 834 (Benefit Enrollment and Maintenance — used by employers, brokers, and plan sponsors to add, change, or terminate member coverage with health plans).

Errors in these files — a missing ISA segment, an incorrect qualifier, a mismatched subscriber ID in an 834, or an invalid ICD-10 code — cause claim rejections or enrollment failures that delay payments or leave members uninsured for weeks. This problem statement asks teams to build a full-featured, open-source X12 EDI parser and validator covering all three transaction types with a clean web UI, AI-assisted error explanation, and actionable fix suggestions.

Problem Statement

Build a web-based application that allows a user to upload any X12 837 (professional or institutional), 835, or 834 EDI file, automatically detect the transaction type, parse the complete hierarchical loop/segment structure, validate it against the relevant X12 implementation guide rules, surface all errors with plain-language explanations, and suggest corrections — with an AI assistant available to answer follow-up questions about the file

Core Features Required

Ingestion: File Upload & Transaction Detection
• Accept .edi, .txt, .dat, or .x12 file uploads via drag-and-drop or file picker.
• Auto-detect ISA/GS/ST segments to identify transaction type: 837P, 837I, 835, or 834.
• Display basic envelope metadata: sender/receiver IDs, interchange date, GS functional group, transaction set count.

Parser Engine: Full Structural Parser
• Parse the complete loop hierarchy per HIPAA implementation guide for 837, 835, and 834.
• Render the parsed tree in an interactive, collapsible UI — each segment expandable to show element-by-element values with human-readable element names.

Validator: Validation Engine
• Mandatory segment/element checks: flag any required segment missing from a loop.
• Element format validation: NPI (10-digit Luhn), ZIP codes, dates (CCYYMMDD), monetary amounts.
• Qualifier validation: NM108 entity type codes, CLM05 facility type + claim frequency, SVC01 CPT/HCPCS format.
• Cross-segment consistency: patient DOB vs. claim date; total claim charge vs. sum of service line charges.
• 835-specific: CAS adjustment reason codes, PR/CO/OA/PI group codes, CLP reconciliation.
• 834-specific: INS maintenance type codes, member relationship codes, subscriber group/policy numbers, date consistency, duplicate member detection.

UX + AI: Error Report with Plain-Language Explanations

• List all errors and warnings with: loop location, segment ID, element position, error code, and a plain-English description.

• AI chat panel (powered by an LLM API) where users can ask contextual questions and get accurate explanations.

Fix Assistant: Correction Suggestions

• For common, deterministic errors, auto-suggest the corrected value. • Allow the user to accept a fix, which updates the parsed structure and re-validates immediately.

835 Feature: Remittance Summary

• Parse all CLP loops and render: claim ID, billed amount, paid amount, patient responsibility, adjustment reasons, and check/EFT reference.

834 Feature: Member Enrollment Summary

• Parse all Loop 2000 member records and display a tabular summary with colourcoded maintenance types.

• COB view, dependent rollup, and family enrollment grouping.

Suggested Technical Stack (only recommendation, not mandatory)

Component	Recommended Approach
Parser	Python (pure, write the parser from scratch for full control); or Node.js
Validation Rules	Encode HIPAA 5010 implementation guide rules as a JSON/YAML rule set; use a rule engine pattern
AI Chat	OpenAI API (GPT-4o), Google Gemini API, or a locally run Ollama model (Mistral/LLaMA)
Backend API	FastAPI (Python) — endpoints for upload, parse, validate, chat
Frontend	React + TypeScript; react-json-tree or custom component for collapsible segment tree; TailwindCSS
Code/Reference Data	CMS NPI lookup (public API), CMS ICD-10 code list, AHA CPT code list, CARC/RARC code lists from WEDI
Deployment	Docker Compose; Vercel/Netlify for frontend, Railway/Render for backend API

Key X12 Concepts — Quick Reference

Term	Meaning
ISA / IEA	Interchange Control Header/Trailer — outermost envelope, identifies sender and receiver
GS / GE	Functional Group Header/Trailer — groups related transaction sets
ST / SE	Transaction Set Header/Trailer — defines one 837, 835, or 834 document
Loop	Hierarchical grouping of segments (e.g., Loop 2300 = one claim; Loop 2000 in 834 = one member)
Segment	A line in the file, e.g., CLM*1234*500***11:B:1*Y*A*Y*I~
Element	Individual field within a segment, separated by * delimiter
NPI	National Provider Identifier — 10-digit number validated by Luhn algorithm
CARC / RARC	Claim / Remark Adjustment Reason Codes — explain why a claim was adjusted or denied
837P / 837I	Professional claims (CMS-1500 form) / Institutional claims (UB-04 form)
835	Electronic Remittance Advice — insurer's payment explanation back to provider
834	Benefit Enrollment & Maintenance — member add/change/terminate records to a health plan
INS Segment	834 subscriber detail: maintenance type, relationship code, employment status
HD Segment	834 Health Coverage segment: coverage type code, insurance line, plan coverage description
DTP Segment	Date/Time Period — used in 834 for coverage effective date, termination date, and eligibility dates
REF Segment	Reference Identification — carries subscriber group numbers, policy numbers, and SSN in 834 loops

Bonus / Differentiator Features

• Batch Processing: Accept a ZIP file containing multiple EDI files of mixed types; generate a consolidated validation report across all files.
• 835-to-837 Reconciliation: Given both a 837 claim file and the corresponding 835 remittance, match claims by ICN/DCN and show payment vs. billed discrepancies sideby-side.
• 834 Change Delta Report: Given two 834 files from consecutive months, automatically diff them and surface net new additions, terminations, and attribute changes per member.
• 834 Eligibility Cross-check: Compare the member roster from an 834 file against an 837 claims file — flag any claim submitted for a terminated or not-yet-effective member.

• Custom Rule Builder: A UI where users can define their own validation rules without code changes.
• Export: Download the parsed structure as JSON, the error report as PDF, the 834 member roster as CSV, or the corrected EDI file with fixes applied.
• Real NPI Validation: Call the CMS NPPES NPI Registry public API to confirm an NPI is active and matches the provider name in the file.

Sample User Stories

• As a medical billing specialist, I want to upload an 837P file and immediately see all validation errors highlighted with explanations so that I can fix them before submitting to the payer.
• As a healthcare developer, I want to browse the full parsed segment tree of any X12 file so that I can understand its structure without reading raw delimited text.
• As a billing manager, I want to upload an 835 remittance and see a summary of which claims were paid in full, partially adjusted, or denied, with reason codes explained in plain English.
• As a benefits administrator, I want to upload our monthly 834 enrollment file and instantly see which employees are being added, changed, or terminated — and whether the file has any errors.
• As a new billing coder, I want to type a question into the AI assistant and get an instant, accurate explanation.

Evaluation Criteria

Criterion	What Judges Look For
Parser Correctness	Accurately parses all loops and segments of valid 837P, 837I, 835, and 834 test files. No loop misidentification.
Validation Coverage	Catches all mandatory-element errors, qualifier errors, and cross-segment inconsistencies — including 834-specific rules.
Error Explanations	Plain-language error descriptions are accurate, specific (include segment + element position), and actionable.
AI Assistant Quality	The LLM-powered chat correctly answers EDI-specific questions across 837, 835, and 834 contexts.
UI / UX	Collapsible segment tree is navigable; error report is scannable; 835 and 834 summaries are clear to a non-technical user.
Innovation	Bonus features (834 delta report, eligibility cross-check, batch processing, custom rules) earn extra weight.
Presentation	Team explains parser design, validation rule architecture, and demonstrates all four transaction types live.

Expected Deliverable (Round 2): A running web app: judges upload four test files — one valid 837P, one malformed 837I (with 5+ deliberate errors), one 835 remittance, and one 834 member enrollment file. The system correctly identifies each type, renders the full parsed tree, lists all errors with accurate explanations, displays the 835 payment summary and the 834 member roster, and the AI assistant correctly answers at least 3 contextual questions.

PROBLEM STATEMENT 02

AI-Powered Translation Studio

Domain: Enterprise AI / NLP

The Core Challenge: Enterprise translation workflows suffer from inconsistent terminology, no translation reuse, source quality issues, disconnected workflows, and no continuous learning. Build an AI-powered translation studio that validates source quality, reuses historical translations via RAG, translates with glossary and style enforcement, and continuously learns from human-approved corrections

Background

Organizations operating across multilingual markets rely on accurate, consistent document translation. The volume of content requiring translation contracts, member communications, regulatory filings, marketing materials is growing rapidly, but translation workflows remain fragmented and inefficient.

Linguists re-translate identical content because there is no centralized translation memory. Source documents arrive with spelling errors, inconsistent formatting, and punctuation issues that propagate into every target language. Modern AI-powered translation platforms have demonstrated the value of features like translation memories, glossary enforcement, style rules, tone control, and customization profiles but most remain third-party services that cannot be trained on an organization's own domain-specific data.

Problem Statement

Enterprise translation workflows suffer from these critical problems:
• Inconsistent Terminology: The same term is translated differently across documents and linguists (e.g., “PM” vs. “P.M.” vs. “p.m.”), leading to confusion and compliance risks.
• No Translation Reuse: Previously approved translations are not surfaced for new projects. Linguists re-translate identical content, wasting time and introducing variation.
• Source Quality Issues: Input documents contain spelling errors, inconsistent punctuation, missing spaces, and formatting inconsistencies that multiply across every target language.
• Disconnected Workflows: Translation, proofreading, approval, and dictionary updates happen in separate tools with no unified audit trail. • No Continuous Learning: Translation models and glossaries remain static with no mechanism to improve from approved corrections.

Your Challenge
Build Translation Studio - an intelligent translation studio that can:

• Upload & Parse Documents: Accept PDF and DOCX files; extract translatable content while preserving structure (headings, tables, lists).
• Validate Source Quality: Spell check, consistency analysis (flag terminology variations), punctuation/grammar validation, and formatting checks. Present issues visually with severity levels; allow manual or AI-assisted resolution.
• Build & Leverage Translation Memory: Store approved translations. Match new segments against the TM using RAG (vector similarity search). Auto-fill exact matches; show fuzzy suggestions; route new segments to the LLM.
• Translate with LLM: Use a Large Language Model to translate new segments. Support multiple target languages (Spanish, Japanese, French, German, etc.).
• Enforce Glossary: Create and manage glossaries per language pair. Constrain the LLM to use approved terms during translation. Detect conflicts with existing entries.
• Support Style & Tone Profiles: Configure tone (Formal, Official, Conversational, Technical, Social), style rules, and bundle everything into reusable profiles per project or team

• Proofreading & Approval: Side-by-side editor for linguists to review, accept, edit, or reject each segment. Optional multi-level approval. Full audit trail.
• Train & Learn Continuously: Import existing bilingual document pairs to seed the TM. After each approval, update the TM, enrich the glossary, and batch corrections for LLM fine-tuning.

Technical Requirements (only recommendation, not mandatory)

• RAG Pipeline: Segment source documents → generate embeddings → vector search against TM → classify matches (Exact/Fuzzy/New) → LLM translation for unmatched segments.
• Glossary & Style Enforcement: The LLM must respect glossary terms and style/tone settings during translation. • Continuous Learning Loop: Approved translations → TM update → glossary enrichment → optional incremental LLM fine-tuning.
• Interactive UI: Web-based interface for document upload, source validation dashboard, side-by-side review editor, glossary management, style profile configuration, and training module.
• Document Structure Preservation: Maintain headings, tables, lists, and formatting through the translation pipeline.

Key Features

Source Quality Validation Engine
• Context-aware spell check with domain-specific terminology support.
• Consistency analysis: flag terminology and formatting variations within the same document.
• Punctuation and grammar validation (missing spaces, double spaces, inconsistent comma usage).
• Formatting checks (date formats, number formats, capitalization patterns).
• Visual issue dashboard with severity classification and batch AI-fix suggestions.
Translation Memory (TM) & RAG
• Stores every approved source–target pair at sentence and phrase level.
• Vector similarity search for exact (100%) and fuzzy (75–99%) matching.

• Smart fuzzy matching: adapts stored translations to new context (grammar, variables).
• Customizable match thresholds per project. TM versioning with rollback.

Glossary & Terminology Management
• Language-pair glossaries with source term, target term, and context notes.
• Import/export in standard formats (TBX) for CAT tool interoperability.
• Glossary enforcement during translation with grammatical context adaptation.
• Conflict detection and auto-suggest from approved translations.

Style Rules & Tone Profiles
• Tone selection: Formal, Official, Conversational, Technical, Social, Friendly, Diplomatic.
• Style rules: predefined (punctuation, capitalization, numbers) and custom free-text rules.
• Style profiles: bundle glossaries, style rules, and TMs into reusable configurations per team/project.
• AI-generated style rules from sample text that represents your brand voice.

Proofreading, Approval & Learning
• Side-by-side editor: linguist reviews AI output vs. source; accept/edit/reject per segment.
• Multi-level approval workflow with full audit trail.
• On approval: TM updated, glossary enriched, corrections batched for LLM finetuning.
• Bulk training: import existing bilingual document pairs to seed TM and fine-tune LLM.

Bonus Features (For Extra Points!)

• Source Document Improvement: AI-powered grammar correction, paraphrasing, and clarity enhancement for the source document before translation.
• Format-Preserved Export: Output translated documents in original format (PDF/DOCX) with layout preserved.
• Real-Time Collaboration: Multiple linguists working on the same document simultaneously.
• CAT Tool Interoperability: Import/export TMX, XLIFF, and TBX files.
• Post-Translation QA: Tag consistency, number accuracy, length validation, crossdocument consistency.
• Analytics Dashboard: TM leverage rate, linguist productivity, quality trends, cost savings estimator.
• Back-Translation Verification: Translate output back to source language for quick quality check.
• API Integration: REST APIs for CI/CD pipeline integration and automated translation workflows.

Evaluation Criteria

Criteria	Description
Innovation & Use of AI	Creative use of RAG + LLM with continuous learning. Integration of glossary, style rules, and TM into a unified customization layer.
Functionality	End-to-end flow from upload to approved, exported translation. Quality of TM matching, glossary enforcement, and translation output.
Usability (UI/UX)	Responsive, simple, user-friendly interface. Intuitive workflow for non-technical linguists.
Scalability	Designed to handle growing document volumes, expanding TMs, and additional language pairs.
Presentation Skills	Clear articulation of problem, solution, and business impact during demo.
Adherence to Timelines	Project completed within the given timeframe, meeting all submission deadlines.

Expected Outcome

An intelligent translation studio that validates source quality, leverages RAG-based translation memory, translates with glossary and style enforcement via LLMs, supports structured proofreading and approval, and continuously learns from every approved translation — reducing turnaround time, eliminating terminology inconsistency, and empowering linguists to focus on nuance rather than repetitive tasks.

PROBLEM STATEMENT 03

AI Email Coordination Assistant

Domain: AI

Background

Professionals spend significant time coordinating meetings, responding to availability requests, and tracking email threads for updates. These repetitive workflows reduce productivity and introduce human delays in scheduling and follow-ups.

While calendar tools exist, there is no autonomous AI-driven email assistant that can:
- Interpret availability discussions,
- Coordinate across multiple participants,
- Schedule meetings,
- And provide contextual updates from ongoing threads.

Problem Statement

Build an AI-powered email assistant that operates through a dedicated email address and acts as a digital executive assistant.
The assistant should:
• Monitor incoming emails.
• Detect scheduling or availability requests.
• Respond autonomously using its own email identity.
• Coordinate time slots across multiple participants.
• Identify common availability.
• Create a Google Calendar meeting.
• Send calendar invitations automatically.

Additionally, if an email asks for an update about an ongoing topic, the assistant should:
• Analyze the email thread.
• Extract the latest available information.
• Respond with a contextual summary. All outbound emails must include a disclaimer stating that the message was sent by an experimental AI email assistant.

Objective

Design and implement an autonomous email coordination agent that reduces manual scheduling effort and intelligently handles email-based workflow conversations.

Key Features

Scheduling & Coordination
• Parse availability information from free-form emails.
• Handle multiple participants providing multiple time slots.
• Compute overlapping availability.
• Resolve conflicts logically.
• Schedule meetings via Google Calendar API.
• Send calendar invites to all participants.

Thread Intelligence
• Read entire email threads.
• Extract latest status information.
• Respond to update requests accurately.
• Maintain conversational context.

Autonomous Identity
• Operate via a dedicated assistant email address.
• Respond directly to participants.
• Append a mandatory AI disclaimer to all outgoing emails.

Technical Requirements
• Email ingestion via SMTP/IMAP or provider APIs.
• Natural language parsing of time expressions.
• Time zone awareness and normalization.
• Integration with Google Calendar APIs.
• Conflict resolution logic.
• Secure authentication and token storage.
• All AI components must use open-source or freely available models where possible.

Bonus Features (For Extra Points!)

• Multi-timezone optimization.
• Suggest optimal meeting times based on historical patterns.
• Priority-based scheduling (e.g., VIP participants).
• Detect ambiguous availability and ask clarifying questions.
• Maintain memory of preferred working hours.
• Human override capability.

Evaluation Criteria

1. Autonomy
   - Ability to coordinate meetings without manual intervention.
2. Accuracy
   - Correct extraction of time slots.
   - Correct identification of overlapping availability.
   - Accurate thread summarization.

3. Reliability
   - No duplicate meetings.
   - Proper email formatting and disclaimers.
4. Security
   - Secure handling of email credentials and tokens.
5. User Experience o Clean configuration interface.
   - Clear scheduling confirmations.

Expected Outcome

A fully functional AI email assistant capable of autonomously coordinating meetings and responding to update requests, significantly reducing manual email overhead.

PROBLEM STATEMENT 04

AI-Powered Autonomous Drone Response for Urban Safety

Domain: AI

Background

As cities grow in scale and complexity, ensuring public safety in real time has become a major challenge. Traditional surveillance systems are passive and rely heavily on manual monitoring, which leads to delayed responses during emergencies such as public disturbances, accidents, unauthorized intrusions, and suspicious activities.

With the evolution of Artificial Intelligence, real-time video analytics, geospatial intelligence, and autonomous aerial systems, it is now possible to build proactive safety platforms that can: • Detect incidents automatically • Dispatch the nearest responder • Provide live situational awareness to control centre This hackathon challenges participants to design a simulation-based AI-powered aerial urban safety system that improves emergency response time and enables smarter city-wide monitoring.

Problem Statement

Build an AI-powered urban safety platform that:
• Detects safety incidents from video feeds using computer vision
• Associates detected incidents with a geolocation (simulated or mapped)
• Automatically dispatches the nearest available drone (simulated)
• Displays real-time drone movement on a map-based command dashboard
• Provides live incident monitoring and status updates The solution must demonstrate a complete working flow from: Video Input → AI Detection → Alert Generation → Drone Dispatch → Live Map Visualization Physical drones are not required. All drone behaviour and movement can be simulated in software.

Key Features

AI-Based Incident Detection (from Video Feeds)
Participants may use live camera input or sample/pre-recorded videos. The system should detect one or more of the following:
• Sudden crowd gathering in a low-density area
• Road accidents or fallen individuals
• Unauthorized entry into restricted zones
• Suspicious vehicle stops
• Abandoned objects
• Traffic rule violations (optional)

Upon detection, the system should:

- Trigger an alert
- Highlight the incident on the map
- Display incident details (type, time, location, source feed)

Note: Face recognition is not required and should not be implemented.

Drone Movement Visualization (Mandatory)
The system must show real-time drone movement, including:
• Current position updates
• Route from drone to incident location
• Estimated time of arrival (ETA)

Drone movement can be simulated using:
• Mobile GPS • Predefined coordinate paths
• Programmatically generated telemetry Smart Navigation & Safety (Simulation-Based)
• Route optimization to the incident location
• No-fly zone avoidance using geofencing
• Basic obstacle or collision avoidance logic (optional)

Command & Control Dashboard
A unified interface that displays:
• Live drone locations on a map
• Active alerts with severity indicators
• Video feed of the responding drone / source
• Drone status, ETA, and battery levels
• Incident timeline and logs

Technical Requirements

Participants should:
• Use free and preferably open-source AI/ML models
• Use simulated video feeds
• Use simulated drone telemetry (mobile GPS, mock APIs, or predefined paths)
• Implement real-time or near-real-time alert generation
• Build an interactive user interface for monitoring and control Cloud usage is allowed but not mandatory.

Bonus Features (For Extra Points!)

• Edge AI (running detection on the drone/mobile device)
• Intelligent alert prioritization
• False positive filtering
• Multi-incident handling with smart drone allocation
• SOS trigger from a mobile application
• Offline or low-bandwidth mode
• Predictive high-risk zone heatmap

Evaluation Criteria

1. Functionality
• Working end-to-end system
• Successful detection → alert → drone dispatch → live tracking flow
• Real-time system behaviour

2. Innovation
• Intelligent fleet coordination
• Smart routing and prioritization logic
• Scalable and modular architecture

3. AI Implementation
• Effective use of pre-trained models
• Relevance and accuracy of detections
• Real-time performance

4. User Interface & Visualization
• Interactive and intuitive dashboard
• Real-time map-based tracking
• Clear incident monitoring experience

5. System Design & Scalability
• Ability to handle multiple drones and incidents
• Clean and extensible architecture

6. Presentation & Demo
• Clear urban safety use case
• Strong working demonstration with live flow

Expected Outcome

A working prototype that demonstrates:
• Real-time AI-based incident detection from video
• Autonomous drone dispatch using simulated telemetry
• Live command center visualization with moving drones
The solution should showcase how intelligent aerial systems can significantly enhance:
• Urban safety
• Emergency response time
• Smart city operations